ASSIGNMENT-13.5

R.SATHWIK

2303A52T01

QUESTION:

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives:

- Identify code smells and inefficiencies in legacy Python scripts.
- Use AI-assisted coding tools to **refactor** for readability, maintainability, and performance.
- Apply **modern Python best practices** while ensuring output correctness.

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the code.
• Instructions:
o Identify global variables used across functions.
o Refactor the code to pass values using function parameters.
o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1

def calculate_interest(amount):

   return amount * rate

print(calculate_interest(1000))
```

• Expected Output:
o Refactored version passing rate as a parameter or using a configuration structure.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
• Focus Areas:
o Readability
o Logical simplification
o Maintainability

Legacy Code:

```
score = 78
if score >= 90:
  print("Excellent")
else:
  if score >= 75:
    print("Very Good")
  else:
    if score >= 60:
      print("Good")
    else:
      print("Needs Improvement")
```

Expected Outcome:
o Flattened logic using guard clauses or a mapping-based approach.

Task 3 (Refactoring Repeated File Handling Code)

- **Task: Use AI to refactor repeated file open/read/close logic.**
- Focus Areas:
  o DRY principle
  o Context managers
  o Function reuse

Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
```

```
print(f.read())

f.close()
```

Expected Outcome:
o Reusable function using with open() and parameters.

Task 4 (Optimizing Search Logic)

- **Task: Refactor inefficient linear searches using appropriate data structures.**
- Focus Areas:
  - o Time complexity
  - o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

    if u == name:

        found = True

print("Access Granted" if found else "Access Denied")
```

Expected Outcome:
o Use of sets or dictionaries with complexity justification.

Task 5 (Refactoring Procedural Code into OOP Design)

- **Task: Use AI to refactor procedural code into a class-based design.**
- Focus Areas:
  - o Object-Oriented principles
  - o Encapsulation

Legacy Code:

```
salary = 50000

tax = salary * 0.2
```

```
net = salary - tax

print(net)
```

Expected Outcome:
o A class like EmployeeSalaryCalculator with methods and attributes.

Task 6 (Refactoring for Performance Optimization)

- **Task: Use AI to refactor a performance-heavy loop handling large data.**
- Focus Areas:
  - o Algorithmic optimization
  - o Use of built-in functions

Legacy Code:

```
total = 0

for i in range(1, 1000000):

   if i % 2 == 0:

      total += i

print(total)
```

Expected Outcome:
o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Task 7 (Removing Hidden Side Effects)

- **Task: Refactor code that modifies shared mutable state.**
- Focus Areas:
  - o Functional-style refactoring
  - o Predictability

Legacy Code:

```
data = []

def add_item(x):

   data.append(x)
```

```
add_item(10)

add_item(20)

print(data)
```

Expected Outcome:
o Refactored function returning values instead of mutating globals.

Task 8 (Refactoring Complex Input Validation Logic)

- **Task: Use AI to simplify and modularize complex validation rules.**
- Focus Areas:
  - o Readability
  - o Testability

Legacy Code:

```
password = input("Enter password: ")

if len(password) >= 8:

  if any(c.isdigit() for c in password):

    if any(c.isupper() for c in password):

      print("Valid Password")

    else:

      print("Must contain uppercase")

  else:

    print("Must contain digit")

else:

  print("Password too short")
```

Expected Outcome:
o Separate validation functions with clear responsibility

CODE:

"""

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

```
Complete Solution for All Tasks
"""

# ==========================================================================
# TASK 1: Refactoring – Removing Code Duplication (Rectangle Calculations)
# ==========================================================================

class Rectangle:
    """
    A class to represent a rectangle and calculate its properties.

    Attributes:
        length (float): The length of the rectangle
        width (float): The width of the rectangle
    """

    def __init__(self, length, width):
        """Initialize rectangle with length and width."""
        self.length = length
        self.width = width

    def area(self):
        """
        Calculate the area of the rectangle.
```

```
        Returns:
            float: The area (length * width)
        """
        return self.length * self.width

    def perimeter(self):
        """
        Calculate the perimeter of the rectangle.

        Returns:
            float: The perimeter (2 * (length + width))
        """
        return 2 * (self.length + self.width)

    def display(self):
        """Display rectangle area and perimeter."""
        print(f"Area of Rectangle: {self.area()}")
        print(f"Perimeter of Rectangle: {self.perimeter()}")

def task_1_demo():
    """Demonstrate Task 1 refactoring."""
    print("\n=== TASK 1: Rectangle Refactoring ===")
    rectangles = [
        Rectangle(5, 10),
        Rectangle(7, 12),
        Rectangle(10, 15)
    ]
    for rect in rectangles:
```

```
    rect.display()


# ==========================================================================
# TASK 2: Refactoring – Extracting Reusable Functions (Price Calculation)
# ==========================================================================


def calculate_total(price, tax_rate=0.18):
    """
    Calculate total price including tax.

    Args:
        price (float): The base price
        tax_rate (float): Tax rate as decimal (default: 0.18)

    Returns:
        float: Total price after tax
    """

    tax = price * tax_rate
    return price + tax

def task_2_demo():
    """Demonstrate Task 2 refactoring."""
    print("\n=== TASK 2: Price Calculation Refactoring ===")
    prices = [250, 500]
```

```
    for price in prices:
        total = calculate_total(price)
        print(f"Total Price: {total}")


# ==========================================================================
# TASK 3: Refactoring Using Classes (Grade Calculator)
# ==========================================================================


class GradeCalculator:
    """
    A class to calculate grades based on student marks.

    This class encapsulates the logic for determining letter grades
    from numerical marks following standard grading criteria.
    """

    def calculate_grade(self, marks):
        """
        Calculate the grade based on marks.

        Args:
            marks (int/float): Student's marks (0-100)

        Returns:
            str: The corresponding grade (Grade A, B, C, D, or Fail)
```

```
        """
        if marks >= 90:
            return "Grade A"
        elif marks >= 80:
            return "Grade B"
        elif marks >= 70:
            return "Grade C"
        elif marks >= 40:
            return "Grade D"
        else:
            return "Fail"


def task_3_demo():
    """Demonstrate Task 3 refactoring."""
    print("\n=== TASK 3: Grade Calculator Refactoring ===")
    calculator = GradeCalculator()
    student_marks = [85, 72, 95, 38, 65]
    for marks in student_marks:
        grade = calculator.calculate_grade(marks)
        print(f"Marks: {marks} → {grade}")


# ==========================================================================
# TASK 4: Refactoring – Converting Procedural Code to Functions (Square)
# ==========================================================================
```

```
def get_input(prompt="Enter number: "):
    """
    Get integer input from user.

    Args:
        prompt (str): The input prompt

    Returns:
        int: The integer input by user
    """
    return int(input(prompt))

def calculate_square(num):
    """
    Calculate the square of a number.

    Args:
        num (int/float): The number to square

    Returns:
        int/float: The square of the number
    """
    return num * num

def display_result(label, value):
    """
    Display a result with label.
```

```
    Args:
        label (str): Description of the result
        value: The value to display
    """
    print(f"{label}: {value}")


def task_4_demo():
    """Demonstrate Task 4 refactoring."""
    print("\n=== TASK 4: Procedural to Functional Refactoring ===")
    num = 5  # Using hardcoded value for demo
    square = calculate_square(num)
    display_result("Square", square)


# ==========================================================================
# TASK 5: Refactoring Procedural Code into OOP (Employee Salary)
# ==========================================================================


class EmployeeSalaryCalculator:
    """
    A class to calculate employee salary and deductions.

    Attributes:
        salary (float): The gross salary of the employee
        tax_rate (float): Tax rate as decimal (default: 0.2)
    """
```

```
def __init__(self, salary, tax_rate=0.2):
    """
    Initialize salary calculator.

    Args:
        salary (float): The gross salary
        tax_rate (float): Tax rate (default: 0.2 or 20%)
    """
    self.salary = salary
    self.tax_rate = tax_rate

def calculate_tax(self):
    """
    Calculate tax amount.

    Returns:
        float: Tax amount
    """
    return self.salary * self.tax_rate

def calculate_net_salary(self):
    """
    Calculate net salary after tax deduction.

    Returns:
        float: Net salary
    """
```

```
        return self.salary - self.calculate_tax()

    def display_salary_breakdown(self):
        """Display detailed salary breakdown."""
        print(f"Gross Salary: {self.salary}")
        print(f"Tax (20%): {self.calculate_tax()}")
        print(f"Net Salary: {self.calculate_net_salary()}")

def task_5_demo():
    """Demonstrate Task 5 refactoring."""
    print("\n=== TASK 5: OOP Salary Calculator ===")
    calculator = EmployeeSalaryCalculator(50000)
    calculator.display_salary_breakdown()

# ==========================================================================
# TASK 6: Optimizing Search Logic (Using Sets)
# ==========================================================================

class AccessControl:
    """
    A class to manage user access control using efficient data structures.

    Uses sets for O(1) lookup time instead of O(n) linear search.
    """
```

```
    def __init__(self, users=None):
        """
        Initialize access control with authorized users.

        Args:
            users (list): List of authorized usernames
        """
        self.authorized_users = set(users) if users else set()

    def check_access(self, username):
        """
        Check if user has access.

        Args:
            username (str): The username to verify

        Returns:
            bool: True if user is authorized, False otherwise
        """
        return username in self.authorized_users

def task_6_demo():
    """Demonstrate Task 6 refactoring."""
    print("\n=== TASK 6: Optimized Search Using Sets ===")
    access = AccessControl(["admin", "guest", "editor", "viewer"])
    test_users = ["admin", "hacker"]
    for user in test_users:
        result = "Access Granted" if access.check_access(user) else "Access Denied"
```

```
    print(f"{user}: {result}")

# ==========================================================================
# TASK 8: Fibonacci Generator
# ==========================================================================

def generate_fibonacci(n):
    """
    Generate Fibonacci sequence up to n terms.

    Args:
        n (int): Number of Fibonacci numbers to generate

    Returns:
        list: List containing first n Fibonacci numbers
    """
    if n <= 0:
        return []
    elif n == 1:
        return [0]

    fib_sequence = [0, 1]
    for i in range(2, n):
        fib_sequence.append(fib_sequence[i-1] + fib_sequence[i-2])
    return fib_sequence
```

```
def task_8_demo():
    """Demonstrate Task 8 refactoring."""
    print("\n=== TASK 8: Fibonacci Generator ===")
    n = 10
    fib_series = generate_fibonacci(n)
    print(f"Fibonacci series up to {n} terms: {fib_series}")


# ==========================================================================
# TASK 9: Twin Primes Checker
# ==========================================================================


def is_prime(n):
    """
    Check if a number is prime.

    Args:
        n (int): The number to check

    Returns:
        bool: True if prime, False otherwise
    """
    if n < 2:
        return False
    if n == 2:
```

```
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True

def is_twin_prime(p1, p2):
    """
    Check if two numbers are twin primes.

    Twin primes are pairs of prime numbers that differ by 2.

    Args:
        p1 (int): First number
        p2 (int): Second number

    Returns:
        bool: True if both are prime and differ by 2, False otherwise
    """
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2

def task_9_demo():
    """Demonstrate Task 9 refactoring."""
    print("\n=== TASK 9: Twin Primes Checker ===")
    test_pairs = [(11, 13), (17, 19), (5, 7), (10, 12)]
    for p1, p2 in test_pairs:
```

```
    result = "Twin Primes" if is_twin_prime(p1, p2) else "Not Twin Primes"
    print(f"({p1}, {p2}): {result}")

# ==========================================================================
# TASK 10: Chinese Zodiac Program
# ==========================================================================

def get_zodiac(year):
  """
  Get the Chinese Zodiac sign for a given year.

  Args:
    year (int): The year to look up

  Returns:
    str: The corresponding Chinese Zodiac sign
  """
  zodiac_signs = [
    "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
    "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
  ]
  return zodiac_signs[year % 12]

def task_10_demo():
  """Demonstrate Task 10 refactoring."""
```

```
    print("\n=== TASK 10: Chinese Zodiac ===")
    test_years = [2000, 2021, 2024]
    for year in test_years:
        zodiac = get_zodiac(year)
        print(f"{year}: {zodiac}")

# ==========================================================================
# TASK 11: Harshad (Niven) Number Checker
# ==========================================================================

def is_harshad(number):
    """
    Check if a number is a Harshad (Niven) number.

    A Harshad number is divisible by the sum of its digits.

    Args:
        number (int): The number to check

    Returns:
        bool: True if Harshad number, False otherwise
    """
    if number <= 0:
        return False
```

```
    digit_sum = sum(int(digit) for digit in str(number))
    return number % digit_sum == 0

def task_11_demo():
    """Demonstrate Task 11 refactoring."""
    print("\n=== TASK 11: Harshad Number Checker ===")
    test_numbers = [18, 19, 21, 12]
    for num in test_numbers:
        result = "Yes" if is_harshad(num) else "No"
        print(f"{num}: {result}")

# ==========================================================================
# TASK 12: Factorial Trailing Zeros
# ==========================================================================

def count_trailing_zeros(n):
    """
    Count trailing zeros in n! (factorial of n).

    Optimized approach: Count multiples of 5 in n!
    (since there are always more factors of 2 than 5)

    Args:
        n (int): Non-negative integer
```

```
    Returns:
        int: Number of trailing zeros in n!
    """
    if n < 0:
        return 0

    count = 0
    power_of_5 = 5
    while power_of_5 <= n:
        count += n // power_of_5
        power_of_5 *= 5
    return count

def task_12_demo():
    """Demonstrate Task 12 refactoring."""
    print("\n=== TASK 12: Factorial Trailing Zeros ===")
    test_numbers = [5, 10, 25, 100]
    for n in test_numbers:
        zeros = count_trailing_zeros(n)
        print(f"{n}! has {zeros} trailing zeros")

# ==========================================================================
# TASK 13: Collatz Sequence Generator
# ==========================================================================
```

```
def generate_collatz_sequence(n):
    """
    Generate the Collatz sequence starting from n.

    Rules:
    - If n is even: next = n / 2
    - If n is odd: next = 3n + 1
    - Continue until reaching 1

    Args:
        n (int): Starting number (must be positive)

    Returns:
        list: The complete Collatz sequence
    """
    if n <= 0:
        return []

    sequence = [n]
    while n != 1:
        n = n // 2 if n % 2 == 0 else 3 * n + 1
        sequence.append(n)
    return sequence

def task_13_demo():
    """Demonstrate Task 13 refactoring."""
    print("\n=== TASK 13: Collatz Sequence Generator ===")
    test_numbers = [6, 1, 27]
```

```
    for num in test_numbers:
        seq = generate_collatz_sequence(num)
        print(f"Collatz({num}): {seq}")

# ==========================================================================
# TASK 14: Lucas Number Sequence
# ==========================================================================

def generate_lucas_sequence(n):
    """
    Generate Lucas sequence up to n terms.

    Lucas sequence starts with 2, 1 and follows: Ln = Ln-1 + Ln-2

    Args:
        n (int): Number of terms to generate

    Returns:
        list: First n Lucas numbers
    """
    if n <= 0:
        return []
    elif n == 1:
        return [2]
```

```
    lucas = [2, 1]
    for i in range(2, n):
        lucas.append(lucas[i-1] + lucas[i-2])
    return lucas


def task_14_demo():
    """Demonstrate Task 14 refactoring."""
    print("\n=== TASK 14: Lucas Number Sequence ===")
    for n in [1, 5, 10]:
        seq = generate_lucas_sequence(n)
        print(f"Lucas({n}): {seq}")


# ==========================================================================
# TASK 15: Vowel & Consonant Counter
# ==========================================================================


def count_vowels_consonants(text):
    """
    Count vowels and consonants in a string.

    Args:
        text (str): Input string to analyze

    Returns:
        tuple: (vowel_count, consonant_count)
```

```
    """
    vowels = "aeiouAEIOU"
    vowel_count = 0
    consonant_count = 0

    for char in text:
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1

    return vowel_count, consonant_count

def task_15_demo():
    """Demonstrate Task 15 refactoring."""
    print("\n=== TASK 15: Vowel & Consonant Counter ===")
    test_strings = ["hello", "aeiou", ""]
    for text in test_strings:
        vowels, consonants = count_vowels_consonants(text)
        print(f"'{text}': Vowels={vowels}, Consonants={consonants}")

# ==========================================================================
# MAIN EXECUTION
# ==========================================================================
```

```
if __name__ == "__main__":
    print("=" * 70)
    print("LAB 13: CODE REFACTORING - COMPLETE SOLUTION")
    print("=" * 70)

    task_1_demo()
    task_2_demo()
    task_3_demo()
    task_4_demo()
    task_5_demo()
    task_6_demo()
    task_8_demo()
    task_9_demo()
    task_10_demo()
    task_11_demo()
    task_12_demo()
    task_13_demo()
    task_14_demo()
    task_15_demo()

    print("\n" + "=" * 70)
    print("All tasks completed successfully!")
    print("=" * 70)
```

OUTPUT:

```
PS C:\Users\ragis\OneDrive\Desktop\ai assistent lab> & C:\Users\ragis\AppData\Local\Python\pythonc
ore-3.14-64\python.exe "c:/Users/ragis/OneDrive/Desktop/ai assistent lab/ass-13.5 (1).py"
======================================================================
LAB 13: CODE REFACTORING - COMPLETE SOLUTION
======================================================================

=== TASK 1: Rectangle Refactoring ===
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50

=== TASK 2: Price Calculation Refactoring ===
Total Price: 295.0
Total Price: 590.0

=== TASK 3: Grade Calculator Refactoring ===
Marks: 85 → Grade B
Marks: 72 → Grade C
Marks: 95 → Grade A
Marks: 38 → Fail
Marks: 65 → Grade D

=== TASK 4: Procedural to Functional Refactoring ===
Square: 25

=== TASK 5: OOP Salary Calculator ===
Gross Salary: 50000
Tax (20%): 10000.0
```

```
Net Salary: 40000.0

=== TASK 6: Optimized Search Using Sets ===
admin: Access Granted
hacker: Access Denied

=== TASK 8: Fibonacci Generator ===
Fibonacci series up to 10 terms: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

=== TASK 9: Twin Primes Checker ===
(11, 13): Twin Primes
(17, 19): Twin Primes
(5, 7): Twin Primes
(10, 12): Not Twin Primes

=== TASK 10: Chinese Zodiac ===
2000: Dragon
2021: Ox
2024: Dragon

=== TASK 11: Harshad Number Checker ===
18: Yes
19: No
21: Yes
12: Yes

=== TASK 12: Factorial Trailing Zeros ===
5! has 1 trailing zeros
10! has 2 trailing zeros
25! has 6 trailing zeros
100! has 24 trailing zeros
```

```
=== TASK 13: Collatz Sequence Generator ===
Collatz(6): [6, 3, 10, 5, 16, 8, 4, 2, 1]
Collatz(1): [1]
Collatz(27): [27, 82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, 322, 161, 484, 242, 121, 364, 18
2, 91, 274, 137, 412, 206, 103, 310, 155, 466, 233, 700, 350, 175, 526, 263, 790, 395, 1186, 593,
1780, 890, 445, 1336, 668, 334, 167, 502, 251, 754, 377, 1132, 566, 283, 850, 425, 1276, 638, 319,
 958, 479, 1438, 719, 2158, 1079, 3238, 1619, 4858, 2429, 7288, 3644, 1822, 911, 2734, 1367, 4102,
 2051, 6154, 3077, 9232, 4616, 2308, 1154, 577, 1732, 866, 433, 1300, 650, 325, 976, 488, 244, 122
, 61, 184, 92, 46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1]

=== TASK 14: Lucas Number Sequence ===
Lucas(1): [2]
Lucas(5): [2, 1, 3, 4, 7]
Lucas(10): [2, 1, 3, 4, 7, 11, 18, 29, 47, 76]

=== TASK 15: Vowel & Consonant Counter ===
'hello': Vowels=2, Consonants=3
'aeiou': Vowels=5, Consonants=0
'': Vowels=0, Consonants=0

======================================================================
All tasks completed successfully!
======================================================================
PS C:\Users\ragis\OneDrive\Desktop\ai assistent lab>
```